TestNG compact cheatsheet

STEP 1 TestNG Framework

1.1 TestNG Foundations: Annotations, Assertions and Test Control

Why a test framework Test classes go in src/test/java . Run them from the IDE (right click, Run), from a testng.xml file, or with mvn test , where the Maven Surefire plugin finds the tests.

A first test class A test method is a public void method marked @Test . It calls the code under test and checks the result with an assertion . If every assertion holds, the test passes.

The annotation lifecycle

Hard and soft assertions

Test attributes Without priorities, TestNG runs test methods of a class in alphabetical order , not in the order they are written. Tests should not depend on that order. When order matters, say so with priority or dependsOnMethods .

Dependencies between tests dependsOnMethods makes a test wait for others and run only if they passed. If a dependency fails, the dependent test is skipped , not failed, so the report shows one real failure instead of a cascade of misleading ones.

TRAP Forgetting `soft.assertAll()` is a classic bug: every check fails silently and the test still passes. Create a new SoftAssert inside each test, never share one between tests.

EXAM Prefer independent tests. Each one should prepare its own state, for example by logging in through an API in @BeforeMethod. Dependencies make sense for real end to end flows, where a later step truly cannot start without the earlier one.

1.2 Suites, Groups, Parameters and Data Providers

The testng.xml suite file testng.xml describes a run without touching the test code: which classes, which methods, in which <test> blocks. One suite holds one or more tests , each test holds classes, and each class can include or exclude methods.

Groups for smoke and regression runs A test can belong to several groups : @Test(groups = {"smoke", "login"}) . The suite file then runs a slice of the project, such as every smoke test except the ones known to be broken. Exclude always wins over include.

Parameters from testng.xml @Parameters({"browser", "baseUrl"}) fills method arguments from <parameter> tags. A parameter in a <test> overrides the same name at suite level, so one suite can run the same class on Chrome and Firefox.

Data providers A data provider is a method that returns rows of arguments. TestNG calls the test once per row and reports each run with its arguments, so a failure names the exact data that broke it.

Data providers that read files A provider can return an Iterator<Object[]> instead of an array, so rows are produced one at a time. It can also take a Method argument to know which test is asking, and live in a separate class shared through dataProviderClass .

Parallel execution

TRAP Configuration methods such as @BeforeMethod and @BeforeGroups belong to no group by default, so a group filter leaves them out. Add `alwaysRun = true` to setup methods, or give them the same groups, or your tests will run without their setup.

TRAP Parallel tests must not share changeable state. A plain `int` counter or a single `WebDriver` field shared by threads gives random failures. Use `AtomicInteger` and concurrent collections for counters, and `ThreadLocal<WebDriver>` so each thread drives its own browser.

1.3 Listeners, Retries, Factories and Framework Design

Listeners A listener is a class TestNG calls on every test event: start, success, failure, skip, and the start and end of each <test> .

Retrying flaky tests An IRetryAnalyzer decides whether a failed test gets another attempt. It suits failures caused by the environment, such as a slow test server or a network blip. It never fixes a real bug: a wrong total fails on every attempt.

@Factory: one class, many instances A data provider repeats one method with different arguments. A factory repeats a whole class: it returns several instances, each built with different constructor data, and every @Test of the class runs on every instance.

Conditional skips Sometimes a test cannot run meaningfully: the payment sandbox is down, or a destructive test must never run outside the QA environment.

A framework: Page Objects and a BaseTest In the Page Object Model , each page of the application gets a class that holds its locators and actions, such as loginAs(user, pass) . Tests talk to pages, never to locators. When a button's id changes, one page class changes and every test keeps working.

Reports, reruns and CI A CI server such as Jenkins or GitHub Actions runs mvn test on every change or every night, publishes testng-results.xml as test results, and keeps the screenshots and HTML reports as build artifacts.

TRAP Retries hide problems if nobody looks at them. Count and report retried tests, keep the limit low (one or two), and treat a test that needs retries every night as a defect in the test or the environment.

EXAM Keep assertions in tests and out of page objects. A page method returns what it sees, and the test decides whether that is right, so the same page can serve positive and negative tests.